SOFTWARE ENGINEERING · CHAPTER 05

Lab-05: Security in Java Web Applications (Spring Security, JWT)

Secure one application, shop-secure, in five steps: observe the default protection and write your first `SecurityFilterChain`, move users and bcrypt hashes into the database, add roles and method security, switch the API to stateless JWT login, then harden it (CSRF strategy, CORS, headers, XSS, secrets, logging). The challenge adds refresh-token rotation, rate limiting, lockout, an audit log and an OWASP checklist.

Spring Security 7.1.0 Spring Boot 4.1.1 JDK 25 · Maven 3.9 ≈ 6 hours 5 tasks + 1 challenge

How to work through this lab. The tasks build on each other inside *one* project. Commit after every task (`git tag task-N`) so the grader can see each step. Keep a file `security-session.md` with every `curl` transcript asked for below. Tick the acceptance checklist as you go (saved in your browser). Open hints only after 10 minutes of being stuck.

0 Setup — 15 min

1. Verify the toolchain:

```
java -version      # 25
mvn -v             # Apache Maven 3.9.x (or use ./mvnw from the generated project)
git --version
curl --version     # or HTTPie / Postman / IntelliJ HTTP client
openssl version    # used to generate the JWT secret
```

2. Generate the project at start.spring.io with these settings, or run the equivalent command:

```
curl https://start.spring.io/starter.zip \
  -d type=maven-project -d language=java -d javaVersion=25 \
  -d bootVersion=4.1.1 -d groupId=edu.itc.se -d artifactId=shop-secure \
  -d name=shop-secure -d packageName=edu.itc.shop \
  -d dependencies=webmvc,security,oauth2-resource-server,data-jpa,h2,validation \
  -o shop-secure.zip && unzip shop-secure.zip -d shop-secure && cd shop-secure
```

Dependencies: *Spring Web MVC, Spring Security, OAuth2 Resource Server, Spring Data JPA, H2 Database, Validation*. `spring-boot-starter-test` and `spring-security-test` come with the security starter's test support; add `org.springframework.security:spring-security-test` (test scope) explicitly if your IDE cannot resolve `SecurityMockMvcRequestPostProcessors`.

3. Initialise git, confirm `.gitignore` excludes `target/`, then add `.env` and `*-local.yml` to it. First commit.
4. Create `src/main/resources/application.yml` with an in-memory H2 database (`jdbc:h2:mem:shop`), `spring.jpa.hibernate.ddl-auto=update`, `spring.h2.console.enabled=false`. Build once: `./mvnw -q verify` must exit with code 0.
5. Target package layout (create packages as you go):

```
edu.itc.shop
├── ShopSecureApplication.java
├── config/      SecurityConfig, PasswordConfig, JwtConfig, CorsConfig,
SecurityProps
├── security/    JpaUserDetailsService, SecurityEventLogger, RequestIdFilter
├── web/         ProductController, OrderController, RegisterController,
│                AuthController, ReviewController, ApiExceptionHandler
│   └── dto/     RegisterRequest, LoginRequest, TokenResponse,
ReviewRequest, ...
├── service/     ProductService, OrderService, AccountService, ReviewService
├── repository/  ProductRepository, OrderRepository, UserAccountRepository,
ReviewRepository
└── domain/      Product, Order, UserAccount, Role, Review
```

1 Default security and your first filter chain

Easy — 40 min

GOAL

See what Spring Security does with zero configuration, then replace it with an explicit `SecurityFilterChain`: public product catalogue, everything else authenticated with HTTP Basic (for now), and consistent 401/403 answers.

STEPS

1. Create `domain/Product` (`id, name, price, stock`), `ProductRepository`, a `ProductService` and `web/ProductController` with `GET /api/products`, `GET /api/products/{id}`, `POST /api/products` and `DELETE /api/products/{id}`. Seed three products with a `CommandLineRunner`.
2. Start the app and copy the line `Using generated security password: …` from the log. Call `GET /api/products` without and with `-u user:<password>`. Record both transcripts

and explain in `README.md` which auto-configuration produced this behaviour (hint: `--debug`, `UserDetailsServiceAutoConfiguration`).

3. Create `config/SecurityConfig` with `@Configuration @EnableWebSecurity` and one `SecurityFilterChain` bean (lambda DSL only): `GET /api/products/** → permitAll()`; `DELETE /api/products/** → hasRole("ADMIN")`; every other `/api/** → authenticated()`; `anyRequest() → denyAll()`. Enable `httpBasic`, disable `formLogin`, and set `SessionCreationPolicy.STATELESS` with CSRF disabled (write a one-line comment stating why this is safe for a Basic/Bearer-only API).
4. Add two temporary in-memory users with `InMemoryUserDetailsManager` and `{noop}` passwords: `admin/admin` (ADMIN) and `alice/alice` (CUSTOMER). They are replaced in Task 2.
5. Create `web/ApiExceptionHandler` (`@RestControllerAdvice`) returning `ProblemDetail` for validation errors and not-found, and configure `exceptionHandling()` with an `authenticationEntryPoint` and an `accessDeniedHandler` that write `application/problem+json` bodies for 401 and 403.
6. Produce the transcript below (200 anonymous, 401 anonymous POST, 403 customer DELETE, 204 admin DELETE) and save it to `security-session.md`. Tag `task-1`.

STARTER CODE

```
package edu.itc.shop.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import
org.springframework.security.config.annotation.web.configuration.EnableWebSecuri
ty;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.provisioning.InMemoryUserDetailsManager;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    SecurityFilterChain api(HttpSecurity http) throws Exception {
        http.authorizeHttpRequests(auth -> auth
                .requestMatchers(HttpMethod.GET, "/api/products/**").permitAll()
```

```
                // TODO: DELETE /api/products/** only for ADMIN
                // TODO: everything else under /api/** authenticated
                .anyRequest().denyAll())
            .httpBasic(Customizer.withDefaults())
            .formLogin(form -> form.disable())
            // CSRF disabled: this chain accepts no cookie-based credentials
(Basic / Bearer only)
            .csrf(csrf -> csrf.disable())
            .sessionManagement(s ->
s.sessionCreationPolicy(SessionCreationPolicy.STATELESS));
        // TODO: exceptionHandling() with problem+json 401 / 403
        return http.build();
    }

    @Bean
    UserDetailsService temporaryUsers() {           // removed in Task 2
        return new InMemoryUserDetailsManager(
            User.withUsername("admin").password("
{noop}admin").roles("ADMIN").build(),
            User.withUsername("alice").password("
{noop}alice").roles("CUSTOMER").build());
    }
}
```

EXPECTED OUTPUT

```
$ curl -i localhost:8080/api/products
HTTP/1.1 200
Content-Type: application/json
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
Cache-Control: no-cache, no-store, max-age=0, must-revalidate

[{"id":1,"name":"USB-C cable","price":7.90,"stock":25}, ...]

$ curl -i -X POST localhost:8080/api/products -H 'Content-Type:
application/json' -d '{"name":"Mouse","price":19.9,"stock":5}'
HTTP/1.1 401
WWW-Authenticate: Basic realm="Realm"
Content-Type: application/problem+json

{"type":"about:blank","title":"Unauthorized","status":401,"detail":"Authenticati
on required","instance":"/api/products"}

$ curl -i -u alice:alice -X DELETE localhost:8080/api/products/1
HTTP/1.1 403
Content-Type: application/problem+json
```

```
{"type":"about:blank","title":"Forbidden","status":403,"detail":"Access
denied","instance":"/api/products/1"}

$ curl -i -u admin:admin -X DELETE localhost:8080/api/products/1
HTTP/1.1 204
```

Show hints

- Rules are evaluated top to bottom, first match wins: put the `GET /api/products/**` rule before the general `/api/**` rule.
- The entry point is a lambda `(request, response, ex) -> {...}`; set status and content type, then write a `ProblemDetail` with an injected `ObjectMapper`. Keep the `WWW-Authenticate` header for 401 so that `curl -u` and browsers behave correctly.
- If the default `/error` dispatch turns into a 401, add `.dispatcherTypeMatchers(DispatcherType.ERROR).permitAll()` as the first rule.
- Set `logging.level.org.springframework.security=TRACE` for the duration of this task; the log shows which filter rejected each request.

ACCEPTANCE CHECKLIST

- [ ] `README.md` quotes the generated password line and names the auto-configuration behind the default behaviour.
- [ ] One `SecurityFilterChain` bean in lambda DSL; no `WebSecurityConfigurerAdapter`, no `and()`; last rule is `denyAll()`.
- [ ] Anonymous GET 200, anonymous POST 401, customer DELETE 403, admin DELETE 204, all recorded in `security-session.md`.
- [ ] 401 and 403 bodies are `application/problem+json`; the CSRF-disable line carries a justification comment.

2 Users and passwords in the database

Easy — 50 min

GOAL

Replace the in-memory users with JPA entities, bcrypt hashes and a `UserDetailsService`; add a public registration endpoint with validation and a password policy.

STEPS

1. Create `domain/Role` (`CUSTOMER, ADMIN`) and `domain/UserAccount` (`id, username unique, passwordHash, enabled, Set<Role> roles` as an eagerly fetched `@ElementCollection`). Make sure `toString()` never prints the hash.
2. Create `repository/UserAccountRepository extends JpaRepository<UserAccount, Long>` with `Optional<UserAccount> findByUsername(String)` and `boolean existsByUsername(String)`.
3. Create `config/PasswordConfig` exposing a `PasswordEncoder` bean from `PasswordEncoderFactories.createDelegatingPasswordEncoder()`. Delete the temporary `InMemoryUserDetailsManager`.
4. Implement `security/JpaUserDetailsService` (see slide 7). Roles are mapped with `.roles(...)`, disabled accounts with `.disabled(true)`.
5. Create `service/AccountService` with `register(username, rawPassword, roles)` (throws `UsernameTakenException` → 409) and a `CommandLineRunner` in `ShopSecureApplication` that seeds `admin` (ADMIN + CUSTOMER) and `alice` (CUSTOMER) if they do not exist. Read the seed passwords from the properties `shop.seed.admin-password` / `shop.seed.customer-password` (defaults acceptable for dev only).
6. Create `web/RegisterController`: `POST /api/register` with `RegisterRequest` (username 3 to 40 chars, `[a-z0-9_.]+`; password 12 to 72 chars containing upper, lower and digit). Success → 201 with `{"username":"..."}`, never the password; validation failure → 400 problem+json with the field errors. Add the endpoint to `permitAll()`.
7. Open the H2 console once (or query via a test) and paste one row of the `users` table into `security-session.md` to show the `{bcrypt}$2a$10$…` hash. Tag `task-2`.

STARTER CODE

```
package edu.itc.shop.web.dto;

import jakarta.validation.constraints.*;

public record RegisterRequest(
        @NotBlank @Size(min = 3, max = 40)
        @Pattern(regexp = "[a-z0-9_.]+", message = "lower-case letters, digits,
_ and . only")
        String username,
        @NotBlank @Size(min = 12, max = 72)
        @Pattern(regexp = "(?=.*\\d)(?=.*[a-z])(?=.*[A-Z]).*",
                message = "must contain an upper-case letter, a lower-case
letter and a digit")
        String password) {}
```

```
@Service
public class AccountService {
    private final UserAccountRepository users;
    private final PasswordEncoder encoder;
    public AccountService(UserAccountRepository users, PasswordEncoder encoder)
{
        this.users = users; this.encoder = encoder;
    }

    @Transactional
    public UserAccount register(String username, String rawPassword, Set<Role>
roles) {
        if (users.existsByUsername(username)) throw new
UsernameTakenException(username);
        // TODO: build the entity with encoder.encode(rawPassword) and save it
        return null;
    }
}
```

EXPECTED OUTPUT

```
$ curl -i -X POST localhost:8080/api/register -H 'Content-Type:
application/json' \
       -d '{"username":"bob","password":"short"}'
HTTP/1.1 400
Content-Type: application/problem+json

{"type":"about:blank","title":"Bad Request","status":400,"detail":"Validation
failed",
 "instance":"/api/register","errors":[{"field":"password","message":"size must
be between 12 and 72"},
 {"field":"password","message":"must contain an upper-case letter, a lower-case
letter and a digit"}]}

$ curl -i -X POST localhost:8080/api/register -H 'Content-Type:
application/json' \
       -d '{"username":"bob","password":"Correct-Horse-9"}'
HTTP/1.1 201
{"username":"bob"}

$ curl -i -u bob:Correct-Horse-9 localhost:8080/api/me
HTTP/1.1 200
{"name":"bob","roles":["ROLE_CUSTOMER"]}

-- users table (H2)
ID  USERNAME  PASSWORD_HASH
ENABLED
```

```
3   bob
{bcrypt}$2a$10$Qm8Yy4p1mJ0k3G0G3rZbUeC7O0rX3n6nT0Ff1c9uYw1n0UQxRk5aG  TRUE
```

Show hints

- With exactly one `UserDetailsService` bean and one `PasswordEncoder` bean, Boot wires the `DaoAuthenticationProvider` for you; nothing else to configure for HTTP Basic.
- Add a small `GET /api/me` endpoint returning `auth.getName()` and the authorities: it makes every later task easier to verify.
- Two `@Pattern` violations on one field appear as two entries; do not deduplicate them in the advice.
- Never log the `RegisterRequest` record: its default `toString()` contains the password. Override it or log only the username.
- H2 console: set `spring.h2.console.enabled=true` only in a `local` profile, and add `/h2-console/**` to that profile's chain with `frameOptions(sameOrigin)`. Do not commit it enabled by default.

ACCEPTANCE CHECKLIST

- [ ] `UserAccount`/`Role` entities, repository and `JpaUserDetailsService` exist; the in-memory users are gone.
- [ ] Stored passwords are `{bcrypt}` hashes produced by the `DelegatingPasswordEncoder`; the hash is never logged or returned.
- [ ] `POST /api/register` enforces the username and password policy with problem+json field errors and answers 409 for a taken username.
- [ ] The `CommandLineRunner` seeds admin and alice idempotently; seed passwords come from properties, not literals.

3 Roles and method security **Medium** **60 min**

GOAL

Add orders that belong to customers and protect the business operations with `@PreAuthorize` rules, including an owner check, then prove every rule with `@WithMockUser` tests.

STEPS

1. Create `domain/Order` (`id, customer (username), productId, quantity, status` with enum `NEW, SHIPPED, CANCELLED`) and `OrderRepository` with `List<Order> findByCustomer(String)`.

2. Create `config/MethodSecurityConfig` with `@EnableMethodSecurity` and a `RoleHierarchy` bean where `ADMIN` implies `CUSTOMER`.
3. Implement `service/OrderService` with these rules:
   - `place(String customerId, PlaceOrder cmd)`: `@PreAuthorize("#customerId == authentication.name")`
   - `history(String customerId)`: `@PreAuthorize("hasRole('ADMIN') or #customerId == authentication.name")`
   - `get(long id)`: `@PostAuthorize("hasRole('ADMIN') or returnObject.customer == authentication.name")`
   - `ship(long id)`: `@PreAuthorize("hasRole('ADMIN')")`
   - `all()`: `@PreAuthorize("hasRole('ADMIN')")`
4. Create `web/OrderController`: `POST /api/orders` (customer taken from `Authentication`, never from the body), `GET /api/orders` (own history; admins may pass `?customer=alice`), `GET /api/orders/{id}`, `POST /api/orders/{id}/ship`. Map `AccessDeniedException` from method security to the same 403 problem+json as the filter chain.
5. Write `OrderServiceSecurityTest` (`@SpringBootTest`, `@MockitoBean OrderRepository`): alice reads her own history (ok), alice reads bob's history (`AccessDeniedException`), admin reads bob's history (ok), alice ships (`AccessDeniedException`), no user at all (`AuthenticationCredentialsNotFoundException`).
6. Write `OrderControllerSecurityTest` (`@WebMvcTest(OrderController.class)` `@Import(SecurityConfig.class)`): anonymous → 401, `@WithMockUser(roles="CUSTOMER")` on `/ship` → 403, `@WithMockUser(roles="ADMIN")` on `/ship` → 204.
7. Record the curl transcript (alice creates an order, bob reads it → 403, admin reads it → 200). Tag `task-3`.

STARTER CODE

```
@Service
public class OrderService {
    private final OrderRepository orders;
    public OrderService(OrderRepository orders) { this.orders = orders; }

    @PreAuthorize("#customerId == authentication.name")
    @Transactional
    public Order place(String customerId, PlaceOrder cmd) {
        return orders.save(new Order(customerId, cmd.productId(),
cmd.quantity()));
    }

    @PreAuthorize("hasRole('ADMIN') or #customerId == authentication.name")
    public List<Order> history(String customerId) {
```

```
            return orders.findByCustomer(customerId);
        }

        // TODO: get(long id) with @PostAuthorize owner check
        // TODO: ship(long id) and all() for ADMIN only
    }


    @SpringBootTest
    class OrderServiceSecurityTest {
        @Autowired OrderService service;
        @MockitoBean OrderRepository orders;

        @Test
        @WithMockUser(username = "alice", roles = "CUSTOMER")
        void aliceCannotReadBobsHistory() {
            assertThrows(AccessDeniedException.class, () -> service.history("bob"));
            verifyNoInteractions(orders);          // rejected BEFORE the method ran
        }
        // TODO: the other four cases
    }
```

EXPECTED OUTPUT

```
$ curl -i -u alice:Alice-pass-123 -X POST localhost:8080/api/orders \
       -H 'Content-Type: application/json' -d '{"productId":2,"quantity":1}'
HTTP/1.1 201
Location: /api/orders/1
{"id":1,"customer":"alice","productId":2,"quantity":1,"status":"NEW"}

$ curl -i -u bob:Correct-Horse-9 localhost:8080/api/orders/1
HTTP/1.1 403
{"type":"about:blank","title":"Forbidden","status":403,"detail":"Access
denied","instance":"/api/orders/1"}

$ curl -i -u admin:Admin-pass-123 localhost:8080/api/orders/1
HTTP/1.1 200
{"id":1,"customer":"alice","productId":2,"quantity":1,"status":"NEW"}

$ ./mvnw -q test -Dtest='Order*SecurityTest'
[INFO] Tests run: 8, Failures: 0, Errors: 0, Skipped: 0
```

Show hints

- `#customerId` refers to the parameter *name*; compile with `-parameters` (the Boot parent does this) or add `@P("customerId")`.

- `@PostAuthorize` runs after the repository call, so `orders.findById` is invoked even for the wrong user; assert on the exception, not on `verifyNoInteractions`, in that test.
- In the controller, add `@ExceptionHandler(AccessDeniedException.class)` to `ApiExceptionHandler` returning the 403 `ProblemDetail`; otherwise the exception surfaces as 500 when thrown inside the MVC layer.
- In the `@WebMvcTest`, the `SecurityConfig` needs a `UserDetailsService` and `PasswordEncoder`: mock them with `@MockitoBean` or import `PasswordConfig` too.
- For a passing "no user" test, run the method outside any `@WithMockUser`; `@SpringBootTest` starts with an empty `SecurityContext`.

ACCEPTANCE CHECKLIST

- [ ] `@EnableMethodSecurity` is on; all five `OrderService` rules are annotations, not `if` statements in the controller.
- [ ] The owner check uses `#customerId == authentication.name` and `returnObject.customer`; the customer of a new order comes from the `Authentication`.
- [ ] Five service-level and three MockMvc security tests pass with `@WithMockUser`; a 401, a 403 and a success case are all covered.
- [ ] Role hierarchy in place (admin passes customer checks); method-level denials produce the same 403 problem+json as URL denials.

4 JWT login and a stateless API **Medium** **75 min**

GOAL

Replace HTTP Basic with a login endpoint that issues an HS256 JWT carrying roles and an expiry, and validate that token on every request as an OAuth2 resource server with `SessionCreationPolicy.STATELESS`.

STEPS

1. Generate a secret and keep it out of git: `export JWT_SECRET=$(openssl rand -base64 48)`. Create `config/SecurityProps` (`@ConfigurationProperties("shop.security")`, record with `@NotBlank @Size(min=32) String jwtSecret` and `Duration accessTokenTtl`) and reference `${JWT_SECRET}` and `15m` in `application.yml`. Enable it with `@ConfigurationPropertiesScan`.
2. Create `config/JwtConfig` with beans `SecretKey` (HmacSHA256 from the secret), `JwtEncoder` (`NimbusJwtEncoder` + `ImmutableSecret`), `JwtDecoder` (`NimbusJwtDecoder.withSecretKey(...).macAlgorithm(HS256)` with

`JwtValidators.createDefaultWithIssuer("shop-api")`), `JwtAuthenticationConverter` mapping the `roles` claim with an empty prefix, and an `AuthenticationManager` (`ProviderManager` over a `DaoAuthenticationProvider`).

3. Create `web/AuthController`: `POST /api/auth/login` with `LoginRequest(username, password)` → `TokenResponse(token, tokenType="Bearer", expiresIn)`. Claims: `iss=shop-api`, `sub`, `iat`, `exp = now + ttl`, `roles` (list of authority strings), `jti` (random UUID). Wrong password → the generic 401 problem+json (catch `AuthenticationException`, never say which part was wrong).
4. Change the filter chain: `permitAll()` for `/api/auth/**` and `/api/register`; remove `httpBasic`; add `oauth2ResourceServer(o -> o.jwt(j -> j.jwtAuthenticationConverter(conv)))`; keep `STATELESS` and CSRF disabled (update the justification comment: Bearer only). Make the 401 entry point also cover the resource server (`o.authenticationEntryPoint(...)`) so expired tokens return problem+json *and* the `WWW-Authenticate: Bearer error="invalid_token"` header.
5. Decode the token you receive at jwt.io (or with `base64 -d`) and paste header and payload into `security-session.md`. Explain in one sentence why the payload is readable and why that is fine.
6. Rewrite `OrderControllerSecurityTest` with the `jwt()` post-processor: `jwt().jwt(j -> j.subject("alice").claim("roles", List.of("ROLE_CUSTOMER"))).authorities(...)`; add a test that a request with header `Authorization: Bearer not.a.jwt` gets 401 problem+json. Write `AuthFlowIT` (`@SpringBootTest(webEnvironment = RANDOM_PORT)`, `@TestPropertySource(properties = "JWT_SECRET=<48 test chars>")`): login → 200 with token; token on `GET /api/orders` → 200; an expired token (issue one with `exp` in the past using the `JwtEncoder` in the test) → 401 with `detail` mentioning expiry.
7. Record the transcript below. Tag `task-4`.

STARTER CODE

```
package edu.itc.shop.config;

@Configuration
public class JwtConfig {

    @Bean
    SecretKey jwtKey(SecurityProps props) {
        return new
SecretKeySpec(props.jwtSecret().getBytes(StandardCharsets.UTF_8), "HmacSHA256");
    }

    @Bean
    JwtEncoder jwtEncoder(SecretKey key) {
        return new NimbusJwtEncoder(new ImmutableSecret<>(key));
```

```
    }

    @Bean
    JwtDecoder jwtDecoder(SecretKey key) {
        NimbusJwtDecoder decoder = NimbusJwtDecoder.withSecretKey(key)
                .macAlgorithm(MacAlgorithm.HS256).build();
        decoder.setJwtValidator(JwtValidators.createDefaultWithIssuer("shop-
api"));
        return decoder;
    }

    @Bean
    JwtAuthenticationConverter jwtAuthenticationConverter() {
        var roles = new JwtGrantedAuthoritiesConverter();
        roles.setAuthoritiesClaimName("roles");
        roles.setAuthorityPrefix("");
        var conv = new JwtAuthenticationConverter();
        conv.setJwtGrantedAuthoritiesConverter(roles);
        return conv;
    }

    @Bean
    AuthenticationManager authenticationManager(UserDetailsService uds,
PasswordEncoder encoder) {
        var provider = new DaoAuthenticationProvider(uds);
        provider.setPasswordEncoder(encoder);
        return new ProviderManager(provider);
    }
}


@RestController
@RequestMapping("/api/auth")
public class AuthController {
    private final AuthenticationManager authManager;
    private final JwtEncoder encoder;
    private final SecurityProps props;
    // constructor

    @PostMapping("/login")
    public TokenResponse login(@RequestBody @Valid LoginRequest req) {
        Authentication auth = authManager.authenticate(

UsernamePasswordAuthenticationToken.unauthenticated(req.username(),
req.password()));
        Instant now = Instant.now();
        JwtClaimsSet claims = JwtClaimsSet.builder()
                .issuer("shop-api").subject(auth.getName())
```

```
                .issuedAt(now).expiresAt(now.plus(props.accessTokenTtl()))
                .id(UUID.randomUUID().toString())
                // TODO: "roles" claim from auth.getAuthorities()
                .build();
        JwsHeader header = JwsHeader.with(MacAlgorithm.HS256).build();
        String token = encoder.encode(JwtEncoderParameters.from(header,
claims)).getTokenValue();
        return new TokenResponse(token, "Bearer",
props.accessTokenTtl().toSeconds());
    }
}
```

EXPECTED OUTPUT

```
$ curl -s -X POST localhost:8080/api/auth/login -H 'Content-Type:
application/json' \
       -d '{"username":"alice","password":"Alice-pass-123"}'
{"token":"eyJhbGciOiJIUzI1NiJ9.eyJpc3MiOiJzaG9wLWFwaSIsInN1YiI6ImFsaWNlIiwiaWF0I
joxNzkwMDAwMDAwLCJleHAiOjE3OTAwMDA5MDAsInJvbGVzIjpbIlJPTEVfQ1VTVE9NRVIiXX0.R6c..
.","tokenType":"Bearer","expiresIn":900}

$ TOKEN=$(curl -s -X POST localhost:8080/api/auth/login -H 'Content-Type:
application/json' \
          -d '{"username":"alice","password":"Alice-pass-123"}' | jq -r .token)
$ curl -i localhost:8080/api/orders -H "Authorization: Bearer $TOKEN"
HTTP/1.1 200
[{"id":1,"customer":"alice","productId":2,"quantity":1,"status":"NEW"}]

$ curl -i localhost:8080/api/orders -H "Authorization: Bearer $TOKEN" -X POST \
       -H 'Content-Type: application/json' -d '{"productId":1,"quantity":2}'
HTTP/1.1 201        # no cookie, no session: check that no Set-Cookie header
appears anywhere

$ curl -i localhost:8080/api/orders -H "Authorization: Bearer $EXPIRED"
HTTP/1.1 401
WWW-Authenticate: Bearer error="invalid_token", error_description="An error
occurred while attempting to decode the Jwt: Jwt expired at 2026-09-
25T09:59:00Z"
Content-Type: application/problem+json

{"type":"about:blank","title":"Unauthorized","status":401,"detail":"Token
expired or invalid","instance":"/api/orders"}

$ curl -i -X POST localhost:8080/api/auth/login -H 'Content-Type:
application/json' \
       -d '{"username":"alice","password":"wrong"}'
HTTP/1.1 401
```

```
{"type":"about:blank","title":"Unauthorized","status":401,"detail":"Bad
credentials","instance":"/api/auth/login"}
```

Show hints

- A short secret produces `KeyLengthException: The secret length must be at least 256 bits`; 48 random bytes base64-encoded give 64 characters, comfortably enough.
- The `JwtDecoder` bean you define wins over the auto-configured one; do not also set `spring.security.oauth2.resourceserver.jwt.*` properties.
- For the expired-token test, build a `JwtClaimsSet` with `expiresAt(now.minusSeconds(120))` (beyond the 60 s clock skew) and encode it with the same `JwtEncoder` bean.
- To tell "expired" from "tampered" in the 401 detail, the `AuthenticationException` passed to your entry point is an `InvalidBearerTokenException` whose message contains "expired". Keep the detail generic otherwise.
- `jwt()` in tests bypasses the `JwtDecoder`, so `@WebMvcTest` still needs a `@MockitoBean JwtDecoder` (or the imported `JwtConfig` plus a test property for the secret).

ACCEPTANCE CHECKLIST

- [ ] `POST /api/auth/login` returns an HS256 JWT with `iss`, `sub`, `iat`, `exp` (15 min), `jti` and `roles`; the secret comes from `JWT_SECRET` and is at least 32 bytes.
- [ ] The API is a resource server: Bearer tokens are validated by `NimbusJwtDecoder`, roles map to authorities, HTTP Basic is gone, no `Set-Cookie` ever appears.
- [ ] Expired or malformed tokens and wrong passwords all answer 401 problem+json (plus `WWW-Authenticate` for tokens) without revealing which part failed.
- [ ] MockMvc tests use the `jwt()` post-processor; `AuthFlowIT` covers login → access → expired token; the decoded token is documented in `security-session.md`.

5 Hardening: CSRF, CORS, headers, XSS, secrets, logging

Hard

90 min

GOAL

Close the gaps around the core: document the CSRF decision, open CORS for one front-end origin only, verify the security headers, demonstrate and defeat a stored XSS attempt in product reviews, move every secret to the environment and log authentication events.

STEPS

1. **CSRF strategy.** Write `docs/csrf.md` (half a page): which credentials each filter chain accepts, why CSRF is disabled for the Bearer-only `/api/**` chain, and how you would configure it (`CookieCsrfTokenRepository.withHttpOnlyFalse()`) if a same-origin SPA with a session cookie were added. Include the fetch snippet that sends `X-XSRF-TOKEN`.
2. **CORS.** Create `config/CorsConfig` with a `CorsConfigurationSource` for `/api/**`: origin `http://localhost:5173`, methods GET/POST/PUT/DELETE, headers `Authorization` and `Content-Type`, exposed header `Location`, max age 1 h. Enable `http.cors(Customizer.withDefaults())`. Verify a preflight with curl (below) and that `Origin: http://evil.example` gets no `Access-Control-Allow-Origin`.
3. **Headers.** Configure `http.headers(...)`: `frameOptions(deny)`, HSTS with `includeSubDomains` and one year, a `Content-Security-Policy` of `default-src 'self'; frame-ancestors 'none'; object-src 'none'`, `Referrer-Policy: strict-origin-when-cross-origin`. Verify with `curl -I` and paste the header block.
4. **Stored XSS demo.** Create `domain/Review`, `ReviewRequest(@NotNull Long productId, @Min(1) @Max(5) int stars, @NotBlank @Size(max=1000) String text)`, `POST /api/reviews` (authenticated) and `GET /api/products/{id}/reviews` (public). Add a tiny server-rendered page `GET /products/{id}/reviews.html` (Thymeleaf, add `spring-boot-starter-thymeleaf`) rendering the reviews with `th:text`. Post the review `<img src=x onerror="document.title='pwned'">` as alice, then show (a) the JSON API returns it escaped by Jackson with `Content-Type: application/json` and `nosniff`, (b) the HTML page shows it as literal text. Then, in a branch you do *not* merge, switch to `th:utext` once and screenshot the difference for `security-session.md`.
5. **Secrets.** Grep the repository: `git grep -n -i -E "password|secret" -- '*.yml' '*.properties' '*.java'`. Every hit must be a placeholder (`${...}`) or a test-only value. Move the seed passwords to `SHOP_SEED_ADMIN_PASSWORD` / `SHOP_SEED_CUSTOMER_PASSWORD`, provide `.env.example` (no values), and make the app fail fast at start-up when `JWT_SECRET` is missing (that is what the `@Validated` record does; prove it with a transcript).
6. **Logging.** Create `security/SecurityEventLogger` with `@EventListener` methods for `AuthenticationSuccessEvent`, `AbstractAuthenticationFailureEvent` and `AuthorizationDeniedEvent`, writing to a logger named `SECURITY`. Register a `SpringAuthorizationEventPublisher` bean so denials are published. Add `security/RequestIdFilter` (`OncePerRequestFilter`, MDC key `rid`, header `X-Request-Id`) before `AuthorizationFilter`, and put `%X{rid}` into the log pattern. Show three log lines: failed login, successful login, access denied.
7. Tag `task-5`. Run `./mvnw -q verify` from a clean clone with only the environment variables from `.env.example` set.

STARTER CODE

```
@Bean
SecurityFilterChain api(HttpSecurity http, JwtAuthenticationConverter conv,
                        ProblemEntryPoint entryPoint) throws Exception {
    http.securityMatcher("/api/**")
        .cors(Customizer.withDefaults())
        // CSRF disabled: this chain accepts Bearer tokens only, never cookies
(see docs/csrf.md)
        .csrf(csrf -> csrf.disable())
        .headers(h -> h
            .frameOptions(f -> f.deny())
            .httpStrictTransportSecurity(hsts ->
hsts.includeSubDomains(true).maxAgeInSeconds(31_536_000))
            .contentSecurityPolicy(csp -> csp.policyDirectives(
                "default-src 'self'; frame-ancestors 'none'; object-src
'none'"))
            .referrerPolicy(r ->
r.policy(ReferrerPolicy.STRICT_ORIGIN_WHEN_CROSS_ORIGIN)))
        .sessionManagement(s ->
s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/api/auth/**", "/api/register").permitAll()
            .requestMatchers(HttpMethod.GET, "/api/products/**").permitAll()
            .anyRequest().authenticated())
        .oauth2ResourceServer(o -> o.authenticationEntryPoint(entryPoint)
                                    .jwt(j ->
j.jwtAuthenticationConverter(conv)))
        .exceptionHandling(e -> e.authenticationEntryPoint(entryPoint))
        .addFilterBefore(new RequestIdFilter(), AuthorizationFilter.class);
    return http.build();
}


@Component
public class SecurityEventLogger {
    private static final Logger log = LoggerFactory.getLogger("SECURITY");

    @EventListener
    public void onFailure(AbstractAuthenticationFailureEvent e) {
        log.warn("login.failed user={} reason={}",
                 e.getAuthentication().getName(),
e.getException().getClass().getSimpleName());
    }
    // TODO: AuthenticationSuccessEvent, AuthorizationDeniedEvent
}

@Bean
AuthorizationEventPublisher
```

```
    authorizationEventPublisher(ApplicationEventPublisher publisher) {
        return new SpringAuthorizationEventPublisher(publisher);
    }
```

EXPECTED OUTPUT

```
$ curl -i -X OPTIONS localhost:8080/api/orders -H 'Origin:
http://localhost:5173' \
       -H 'Access-Control-Request-Method: POST' -H 'Access-Control-Request-
Headers: authorization,content-type'
HTTP/1.1 200
Access-Control-Allow-Origin: http://localhost:5173
Access-Control-Allow-Methods: GET,POST,PUT,DELETE
Access-Control-Allow-Headers: authorization, content-type
Access-Control-Max-Age: 3600
Vary: Origin

$ curl -I localhost:8080/api/products | grep -iE 'x-frame|strict-
transport|content-security|referrer|x-content'
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
Strict-Transport-Security: max-age=31536000 ; includeSubDomains
Content-Security-Policy: default-src 'self'; frame-ancestors 'none'; object-src
'none'
Referrer-Policy: strict-origin-when-cross-origin

$ curl -s localhost:8080/api/products/2/reviews
[{"id":1,"author":"alice","stars":1,"text":"<img src=x
onerror=\"document.title='pwned'\">"}]
-- rendered with th:text the page source shows:
<p>&lt;img src=x onerror=&quot;document.title=&#39;pwned&#39;&quot;&gt;</p>

$ JWT_SECRET= ./mvnw -q spring-boot:run
***************************
APPLICATION FAILED TO START
***************************
Binding to target edu.itc.shop.config.SecurityProps failed:
    Property: shop.security.jwt-secret   Reason: must not be blank

10:14:02.117 WARN  rid=7f3c9a12 SECURITY : login.failed user=alice
reason=BadCredentialsException
10:14:05.402 INFO  rid=8a10c0de SECURITY : login.ok user=alice
type=UsernamePasswordAuthenticationToken
10:15:31.008 WARN  rid=91be77f0 SECURITY : access.denied user=alice
object=OrderService.ship(1)
```

Show hints

- HSTS is only written for HTTPS requests by default; to see it over plain `localhost` during the lab add `.requestMatcher(AnyRequestMatcher.INSTANCE)` to the HSTS config and note in the README that production uses the default.
- The Thymeleaf page needs its own chain (`@Order(2)`, `securityMatcher("/products/**")`, `permitAll()` for GET); the `/api/**` chain does not match it at all once it has a `securityMatcher`. Confirm which chain handled the request with TRACE logging.
- Do not register `RequestIdFilter` as a `@Component`; instantiate it in the chain only, otherwise it runs twice.
- Log pattern: `logging.pattern.console=%d{HH:mm:ss.SSS} %-5level rid=%X{rid} %logger{0} : %msg%n`.
- Your own `ProviderManager` does not publish events by itself: inject Boot's `AuthenticationEventPublisher` bean into `JwtConfig` and call `providerManager.setAuthenticationEventPublisher(publisher)`, otherwise the login endpoint's successes and failures never reach `SecurityEventLogger`.

ACCEPTANCE CHECKLIST

- [ ] `docs/csrf.md` explains the per-chain decision; CORS allows exactly `http://localhost:5173` and the preflight transcript plus the rejected evil origin are recorded.
- [ ] `curl -I` shows nosniff, `X-Frame-Options: DENY`, HSTS, CSP with `frame-ancestors 'none'` and a Referrer-Policy.
- [ ] The XSS payload is stored unchanged, validated for size, and rendered harmlessly by both the JSON API and the `th:text` page; the `th:utext` counter-example is documented.
- [ ] `git grep` finds no literal secret; start-up fails without `JWT_SECRET`; the `SECURITY` logger shows failed login, successful login and access denied with request ids.

★

Challenge: refresh tokens, lockout, audit log and an OWASP checklist

Hard **120 min · optional, extra credit**

SCENARIO

The shop goes live. Access tokens must stay short-lived without forcing customers to log in every 15 minutes, a stolen refresh token must be detectable, brute-force logins must be slowed down and locked out, every security-relevant event must be kept in an audit table, and the team lead wants a one-page OWASP Top 10 checklist with evidence before the release.

REQUIREMENTS

1. **Refresh tokens with rotation.** `POST /api/auth/login` additionally returns an opaque `refreshToken` (32 random bytes, base64url). Store only its SHA-256 hash in a `refresh_tokens` table (`id, tokenHash, username, familyId, expiresAt (7 days), revokedAt, replacedBy`). `POST /api/auth/refresh` exchanges a valid refresh token for a new access token *and* a new refresh token, revoking the old one. Presenting an already-rotated token revokes the whole `familyId` (theft detected). `POST /api/auth/logout` revokes the family.
2. **Rate limiting.** Limit `/api/auth/login` to 5 attempts per minute per IP and per username with Bucket4j (`com.bucket4j:bucket4j-core`) or a simple `ConcurrentHashMap` counter with expiry; excess → 429 problem+json with `Retry-After`.
3. **Account lockout.** After 5 consecutive failures lock the account for 15 minutes (`failedAttempts`, `lockedUntil` on `UserAccount`, driven by the authentication events). A locked account yields the same generic 401 as a wrong password (no enumeration), but the audit log records `LOCKED`. Successful login resets the counter.
4. **Audit log.** `security_audit` table (`at, type, username, ip, requestId, detail`) written by an `AuditService` with `@Transactional(propagation = REQUIRES_NEW)` for: LOGIN_OK, LOGIN_FAILED, LOCKED, TOKEN_REFRESHED, REFRESH_REUSE_DETECTED, LOGOUT, ACCESS_DENIED, ROLE_CHANGED. Expose `GET /api/admin/audit?username=` for ADMIN only.
5. **OWASP checklist.** `docs/owasp-checklist.md`: a table with the ten 2021 categories, the control in this project (class or config line) and the evidence (test name or transcript). Categories without a control (A06, A10) must state the gap and the planned mitigation.
6. **Integration suite.** `AuthLifecycleIT` (`@SpringBootTest(webEnvironment = RANDOM_PORT)`) covering: login → access → refresh (old refresh token now rejected) → access with the new token → reuse of the old refresh token → whole family revoked → 401 on the newest token's refresh; plus lockout after 5 failures and a 429 from the rate limiter.

SKELETON

```
@Service
public class RefreshTokenService {
    private static final Duration TTL = Duration.ofDays(7);
    private final RefreshTokenRepository repo;
    private final SecureRandom random = new SecureRandom();
    // constructor

    public record Issued(String rawToken, RefreshToken entity) {}

    @Transactional
```

```
    public Issued issue(String username, UUID family) {
        byte[] bytes = new byte[32];
        random.nextBytes(bytes);
        String raw =
Base64.getUrlEncoder().withoutPadding().encodeToString(bytes);
        RefreshToken t = new RefreshToken(sha256(raw), username, family,
Instant.now().plus(TTL));
        return new Issued(raw, repo.save(t));
    }

    @Transactional
    public Issued rotate(String raw) {
        RefreshToken current = repo.findByTokenHash(sha256(raw))
                .orElseThrow(() -> new InvalidRefreshTokenException("unknown"));
        if (current.isRevoked()) {
            // TODO: reuse detected → revoke the whole family, audit
REFRESH_REUSE_DETECTED
            throw new InvalidRefreshTokenException("reuse");
        }
        if (current.isExpired()) throw new
InvalidRefreshTokenException("expired");
        Issued next = issue(current.getUsername(), current.getFamilyId());
        current.revoke(next.entity().getId());
        return next;
    }

    static String sha256(String s) {
        try {
            return HexFormat.of().formatHex(
                MessageDigest.getInstance("SHA-
256").digest(s.getBytes(StandardCharsets.UTF_8)));
        } catch (NoSuchAlgorithmException e) { throw new
IllegalStateException(e); }
    }
}


// Lockout driven by events, not by the controller
@Component
public class LockoutListener {
    @EventListener
    @Transactional
    public void onFailure(AuthenticationFailureBadCredentialsEvent e) {
        users.findByUsername(e.getAuthentication().getName()).ifPresent(u -> {
            u.registerFailure();                        // attempts++, lock at 5
for 15 min
            audit.record(u.isLocked() ? AuditType.LOCKED :
AuditType.LOGIN_FAILED, u.getUsername());
```

```
        });
    }
    @EventListener
    @Transactional
    public void onSuccess(AuthenticationSuccessEvent e) { /* TODO reset counter,
audit LOGIN_OK */ }
}
```

EXPECTED OUTPUT

```
$ curl -s -X POST localhost:8080/api/auth/login -d
'{"username":"alice","password":"Alice-pass-123"}' -H 'Content-Type:
application/json'
{"token":"eyJ...","tokenType":"Bearer","expiresIn":900,"refreshToken":"kX9x...",
"refreshExpiresIn":604800}

$ curl -s -X POST localhost:8080/api/auth/refresh -d
'{"refreshToken":"kX9x..."}' -H 'Content-Type: application/json'
{"token":"eyJ...","tokenType":"Bearer","expiresIn":900,"refreshToken":"Qm2v...",
"refreshExpiresIn":604800}

$ curl -i -X POST localhost:8080/api/auth/refresh -d
'{"refreshToken":"kX9x..."}' -H 'Content-Type: application/json'
HTTP/1.1 401
{"type":"about:blank","title":"Unauthorized","status":401,"detail":"Refresh
token invalid","instance":"/api/auth/refresh"}
-- security_audit: REFRESH_REUSE_DETECTED alice family=5c1e... ; all tokens of
the family revoked

$ for i in 1 2 3 4 5 6; do curl -s -o /dev/null -w '%{http_code} ' -X POST
localhost:8080/api/auth/login \
      -H 'Content-Type: application/json' -d
'{"username":"alice","password":"nope"}'; done
401 401 401 401 401 429

$ ./mvnw -q verify
[INFO] Tests run: 31, Failures: 0, Errors: 0, Skipped: 0
```

Show hints

- Never store or log the raw refresh token; look it up by hash. A leaked database then gives an attacker nothing usable.
- Put the rate limiter in a `OncePerRequestFilter` registered `addFilterBefore(..., BearerTokenAuthenticationFilter.class)` and restricted to `POST /api/auth/login`; Bucket4j: `Bucket.builder().addLimit(Bandwidth.builder().capacity(5).refillGreedy(5, Duration.ofMinutes(1)).build()).build()`, one bucket per key in a `ConcurrentHashMap`.

- Behind a proxy the client IP is in `X-Forwarded-For`; use `ForwardedHeaderFilter` or `server.forward-headers-strategy=framework`, and only trust it from your own proxy.
- The lockout check belongs in `UserDetails.isAccountNonLocked()`; Spring then throws `LockedException`, which your entry point maps to the same 401 body as bad credentials.
- In `AuthLifecycleIT` use `TestRestTemplate` and one `@Test` per scenario step order is important; set `@TestMethodOrder(OrderAnnotation.class)` or write one long test with clear comments.

ACCEPTANCE CHECKLIST

- [ ] Refresh tokens are opaque, stored hashed, rotated on every use; reuse revokes the family; logout revokes the family; all verified by `AuthLifecycleIT`.
- [ ] Login is rate-limited (429 with `Retry-After`) and accounts lock after 5 failures for 15 minutes without revealing the lock to the client.
- [ ] The `security_audit` table records all eight event types in `REQUIRES_NEW` transactions and is readable by ADMIN only.
- [ ] `docs/owasp-checklist.md` maps all ten categories to a control and evidence, or an explicit gap with a plan.

Grading rubric

Component	Points	Full marks when…
Task 1 · Default security & filter chain	10	Lambda-DSL chain with deny-by-default, problem+json 401/403, README explains auto-configuration, transcript complete.
Task 2 · Users and passwords	15	JPA-backed `UserDetailsService`, delegating bcrypt encoder, validated registration with password policy, idempotent seeding from properties.
Task 3 · Roles and method security	20	All five rules as annotations incl. owner checks, role hierarchy, eight passing security tests covering 401/403/200.
Task 4 · JWT login and stateless API	20	Correct claims and expiry, secret from the environment, resource-server validation, generic 401s, `jwt()` tests and end-to-end flow incl. expired token.
Task 5 · Hardening	25	CSRF document, single-origin CORS, verified headers, XSS demo defeated by encoding, no secrets in git, security event logging with request ids.

Component	Points	Full marks when…
Code quality	10	Layered packages, constructor injection, no business rules in controllers, meaningful commit history with tags per task, README and `security-session.md` complete.
★ Challenge (extra credit)	+20	All four acceptance items ticked; `AuthLifecycleIT` green; OWASP checklist with evidence.
Total	**100 (+20)**	

Automatic deductions: −15 for any secret, seed password or JWT key committed to git (even if later removed); −10 for passwords stored with `{noop}`, MD5 or SHA after Task 2; −10 for `permitAll()` on `/**` or `anyRequest().permitAll()`; −10 for CSRF disabled without the written justification or on a cookie-based chain; −5 per role check written as an `if` in a controller; −5 for `hasRole("ROLE_…")` (double prefix); −5 for passwords, tokens or hashes appearing in logs or responses.

Submission

1. Repository layout:

```
shop-secure/
├── pom.xml  ·  mvnw  ·  .mvn/
├── README.md                ← auto-configuration answers, how to run,
environment variables
├── security-session.md      ← every curl transcript, decoded JWT, header
block, XSS screenshots
├── .env.example             ← variable names only: JWT_SECRET,
SHOP_SEED_ADMIN_PASSWORD, ...
├── docs/
│   ├── csrf.md              ← Task 5 CSRF strategy
│   └── owasp-checklist.md   ← challenge only
└── src/
    ├── main/java/edu/itc/shop/…   (config · security · web · service ·
repository · domain)
    ├── main/resources/  application.yml (placeholders only) ·
templates/reviews.html · logback-spring.xml (optional)
    └── test/java/edu/itc/shop/…   OrderServiceSecurityTest ·
OrderControllerSecurityTest · AuthFlowIT · AuthLifecycleIT
```

2. `JWT_SECRET=$(openssl rand -base64 48) ./mvnw -q verify` must succeed from a clean checkout on JDK 25; the tests themselves must not depend on any other environment

variable.

3. Tags `task-1` … `task-5` (and `challenge`) must exist. Push branch `lab-05` and open a merge request titled `Lab-05 – <your name>`.
4. The merge request description lists the environment variables needed to run the app and links to `security-session.md`.

Back to Chapter 05 slides | Retake the chapter quiz | ← Lab-04